

GitLab Usage Guide for QA & SDETs: IntelliJ / Git Bash / SourceTree

Overview

This document provides a **complete step-by-step guide** for using GitLab effectively across our QA and development teams. It covers **branching strategy**, **IntelliJ usage**, **Git Bash commands**, **SourceTree tips**, **naming conventions**, **merge protocols**, and **common FAQs**.

Repository Workflow: High-Level Steps

1. **Start from Main:**
 - Ensure your local main is **up-to-date** with GitLab remote.
 - Pull latest changes: `git pull origin main`
2. **Create a Feature Branch:**
 - Naming convention:
 - `feature/QA-131`
 - `feature/QA-131-forgot-password-idp-login`
 - `QA-131` (invalid – must start with `feature/`)
 - Create branch off main.
3. **Work on the Branch:**
 - Do NOT commit to main directly.
 - Commit regularly with clear messages.
4. **Push to GitLab:**
 - Push feature branch: `git push origin feature/QA-131`
5. **Create a Merge Request (MR):**
 - Target = main
 - Check: Delete source branch after merge
6. **Post-Merge Cleanup:**
 - Switch back to main
 - Pull latest changes
 - Create a new branch for the next story

Best Practices

- Always **rebase off main** before starting new work.
- Never stash and create a branch from dirty state.
- Commit in small, logical chunks.
- Push often to avoid conflicts.
- Always delete your branch after MR is merged.

IntelliJ GitLab Workflow

Clone the Repo

1. VCS > Get from Version Control
2. Paste GitLab HTTPS or SSH URL

Create Branch from Main

1. Checkout main
2. Pull latest: Git > Pull
3. Git > New Branch from Selected...
4. Name it as `feature/QA-131`

Commit & Push

1. Git > Commit...
2. Add message, then **Commit and Push**

Merge Request

1. Open GitLab in browser
2. Create MR from feature/QA-131 main
3. Select "Delete source branch after merge"

After Merge

1. Switch to main
2. Pull
3. Create next feature branch from updated main

Git Bash Commands

```
# Clone repo
git clone git@gitlab.com:your-group/your-repo.git

# Create feature branch
git checkout main
git pull origin main
git checkout -b feature/QA-131

# Commit
git add .
git commit -m "[QA-131] Forgot password flow"

# Push
git push origin feature/QA-131

# After merge
git checkout main
git pull origin main
```

SourceTree Users

- Download: <https://www.sourcetreeapp.com/>
- GUI alternative to IntelliJ or Git Bash.
- Recommended for visual tracking of branches, stashes, merges.

Visual Git Flow Diagram

```
main

feature/QA-131-login-fix
commit
push merge request merged deleted

feature/QA-132-api-coverage
commit
push merge request merged deleted

[ main always up-to-date ] new branch feature/QA-133 repeat
```

This cycle ensures every story/sub-task has:

- An isolated branch
- Proper tracking
- Easier CI/CD traceability

Special Scenarios

Accidentally Started Work on Main

```
git stash

# Switch to main and pull
git checkout main
git pull

# Create correct feature branch
git checkout -b feature/QA-131

# Apply stash
git stash pop
```

Manual Merge Needed (Conflicts)

- In IntelliJ: View Git > Merge Conflicts
- Resolve manually using IntelliJ merge tool
- Then commit and push

Frequently Asked Questions (FAQs)

Q: Can I work on multiple branches?

Yes, but always rebase off updated main for each.

Q: I forgot to check 'delete branch after merge'. What now?

Go to GitLab > Repository > Branches > Delete manually.

Q: Git says 'Your branch is ahead of origin'. What does it mean?

You have local commits not pushed yet. Run: `git push origin your-branch-name`

Q: How do I resolve merge conflicts in IntelliJ?

IntelliJ shows a merge dialog. Resolve, test, then commit.

Q: Can I merge using SourceTree?

Yes – use the "Merge" button, select main as target, resolve conflicts, then push.

Summary Checklist

Always rebase from main
Use correct branch naming (feature/QA-123)
Use IntelliJ or Git Bash consistently
Push feature branches early and often
Delete branches post-MR
Follow clean merge request practices